Movie Revenue Prediction

Ohenmaa Sarfo-Bonsu

2025-12-16

```r
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```r
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.4.3

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(readxl)
library(lubridate)
```

```
## Warning: package 'lubridate' was built under R version 4.4.3

##
## Attaching package: 'lubridate'

## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```r
library(stringr)
library(shiny)
```

```
## Warning: package 'shiny' was built under R version 4.4.3
```

```r
library(rlang)
```

```
## Warning: package 'rlang' was built under R version 4.4.3
```

```r
library(leaps)
library(data.table)
```

```
## Warning: package 'data.table' was built under R version 4.4.3

##
## Attaching package: 'data.table'

## The following object is masked from 'package:rlang':
##
##     :=

## The following objects are masked from 'package:lubridate':
##
##     hour, isoweek, isoyear, mday, minute, month, quarter, second, wday,
##     week, yday, year

## The following objects are masked from 'package:dplyr':
##
##     between, first, last
```

```r
library(GGally)
library(car)
```

```
## Warning: package 'car' was built under R version 4.4.3

## Loading required package: carData

## Warning: package 'carData' was built under R version 4.4.3

##
## Attaching package: 'car'

## The following object is masked from 'package:dplyr':
##
##     recode
```

```r
library(tibble)
```

```
## Warning: package 'tibble' was built under R version 4.4.3
```

```r
library(knitr)
```

```
## Warning: package 'knitr' was built under R version 4.4.3
```

```r
library(kableExtra)
```

```
##
## Attaching package: 'kableExtra'

## The following object is masked from 'package:dplyr':
##
##     group_rows
```

```r
# table of original variables
library(tibble)
library(knitr)
library(kableExtra)

variable_table <- tibble(
  `Factor name` = c(
    "movie_title",
    "production_date",
    "genres",
    "runtime_minutes",
    "director_name",
    "birth_year",
    "movie_averageRating",
    "budget",
    "gross"
  ),
  Description = c(
    "The name of the movie",
    "The day, month, and year that the movie was produced",
    "An alphabetical list of up to three genres that described the movie",
    "Length of the movie in minutes",
    "Name of the director",
    "The year the director was born",
    "A quality rating given to the movie by viewers on a 1-10 scale, with 10 being the best",
    "The movie production budget in (nominal) dollars",
    "The movie's worldwide gross revenue in (nominal) dollars"
  )
)

kable(
  variable_table,
  booktabs = FALSE
) |>
  kable_styling(full_width = FALSE) |>
  column_spec(1, border_right = TRUE) |>
  footnote(
    general = "Figure 1: Data table factor names and descriptions",
    general_title = ""
  )
```

Factor name	Description

movie_title	The name of the movie
production_date	The day, month, and year that the movie was produced
genres	An alphabetical list of up to three genres that described the movie
runtime_minutes	Length of the movie in minutes
director_name	Name of the director
birth_year	The year the director was born
movie_averageRating	A quality rating given to the movie by viewers on a 1–10 scale, with 10 being the best
budget	The movie production budget in (nominal) dollars
gross	The movie's worldwide gross revenue in (nominal) dollars

Figure 1: Data table factor names and descriptions

```r
# read in the dataset
movie <- read_xlsx("dataset89.xlsx")
```

Data Clean-Up and Table editing

Checking for N/A values

```r
sum(is.na(movie))
```

```
## [1] 0
```

Initial check of model consistency

```r
summary(movie)
```

```
##  movie_title        production_date       genres          runtime_minutes
##  Length:103         Length:103         Length:103         Min.   : 80.0
##  Class :character   Class :character   Class :character   1st Qu.: 96.0
##  Mode  :character   Mode  :character   Mode  :character   Median :105.0
##                                                           Mean   :107.8
##                                                           3rd Qu.:115.0
##                                                           Max.   :242.0
##  director_name        birth_year   movie_averageRating     budget
##  Length:103         Min.   :1927   Min.   :3.900       Min.   :   600000
##  Class :character   1st Qu.:1947   1st Qu.:5.800       1st Qu.: 13500000
##  Mode  :character   Median :1960   Median :6.400       Median : 27000000
##                     Mean   :1958   Mean   :6.353       Mean   : 37427184
##                     3rd Qu.:1968   3rd Qu.:7.000       3rd Qu.: 45000000
##                     Max.   :1980   Max.   :8.300       Max.   :170000000
##      gross
##  Min.   :   102812
##  1st Qu.: 25400534
##  Median : 58011485
##  Mean   :101722204
##  3rd Qu.:137804220
##  Max.   :886742787
```

Transformations and Adding Variables That Could be Helpful for Analysis

I added numeric and categorical variables that I thought would be helpful in aiding my analysis. **director_age**: calculated from the current year minus the birth year. **director_age_group**: With director_age, I made an age group I could group the directors in, in case a director's age group has any impact on the revenue. **production_date** was converted to a **date** type, then **month** and **day** variables were added in order to calculate the **production_season** for each movie. **production_day_number** was added, which counts the number of days that elapsed between the production day and the default R date, January 1, 1970.

The **director_freq** categorical variable groups the directors based on how frequently they appeared in the sample. Directors who appeared more than three times vs directors who appeared three or less times.

Since both budget and gross scaled up to the millions, I took the natural log of both variables, and saved them as **log_budget** and **log_gross**, respectively.

```
movie <- movie |>
mutate(
production_date = as.Date(production_date, format = "%m/%d/%y"),
director_age = as.numeric(format(Sys.Date(), "%Y")) - birth_year,
month = as.numeric(format(production_date, "%m")),
day = as.numeric(format(production_date, "%d")),
    production_season = case_when(
      (month == 12 & day >= 21) | (month %in% c(1, 2)) | (month == 3 & day < 20) ~ "Winter",
      (month == 3  & day >= 20) | (month %in% c(4, 5)) | (month == 6 & day < 21) ~ "Spring",
      (month == 6  & day >= 21) | (month %in% c(7, 8)) | (month == 9 & day < 23) ~ "Summer",
      TRUE ~ "Fall"
    ),
production_day_number = as.numeric(production_date),
director_age_group = case_when((director_age >= 40 & director_age < 50) ~ "40s",
                               (director_age >= 50 & director_age < 60) ~ "50s",
                               (director_age >= 60 & director_age < 70) ~ "60s",
                               (director_age >= 70 & director_age < 80) ~ "70s",
                               (director_age >= 80 & director_age < 90) ~ "80s",
                               (director_age >= 90 & director_age < 100) ~ "90s",

                               ),
log_gross = log(gross),
log_budget = log(budget)

)
```

```
# creating a list of directors who appeared more than 3 times, then a list of directors who appeared 3

director_freq <- movie |>
  count(director_name, name = "count") |>
  mutate(mean_count = mean(count),
         min_count = min(count),
         max_count = max(count),
         median_count = median(count),
         frequency = case_when(
           (count > 3 ~ "more than three movies"),
           (count <= 3 ~ "three or less movies")
         )
         )
```

```r
more_than_three_table <- director_freq |>
  filter(frequency == "more than three movies")

more_than_three_list <- more_than_three_table$director_name

three_or_less_table <- director_freq |>
  filter(frequency == "three or less movies")

three_or_less_list <- three_or_less_table$director_name


# add the frequency of the directors to the main data table

movie <- movie |>
  mutate(director_freq = case_when(
    (director_name %in% more_than_three_list  ~ "more than three movies"),
    (director_name %in% three_or_less_list  ~ "three or less movies"),
    (TRUE                                  ~ NA)
         ))




movie
```

```
## # A tibble: 103 x 18
##    movie_title   production_date genres runtime_minutes director_name birth_year
##    <chr>         <date>          <chr>            <dbl> <chr>              <dbl>
##  1 Alita: Battl~ 2019-02-01      Actio~             122 Robert Rodri~       1968
##  2 Alice Throug~ 2016-05-20      Adven~             113 James Bobin         1972
##  3 Harry Potter~ 2005-11-18      Adven~             157 Mike Newell         1942
##  4 Thor          2011-04-29      Actio~             115 Kenneth Bran~       1960
##  5 Gods of Egypt 2016-02-24      Actio~             127 Alex Proyas         1963
##  6 The Green Ho~ 2011-01-14      Actio~             119 Michel Gondry       1963
##  7 I, Robot      2004-07-16      Actio~             115 Alex Proyas         1963
##  8 Starship Tro~ 1997-11-07      Actio~             129 Paul Verhoev~       1938
##  9 Cinderella    2015-03-06      Adven~             105 Kenneth Bran~       1960
## 10 Hollow Man    2000-08-04      Actio~             112 Paul Verhoev~       1938
## # i 93 more rows
## # i 12 more variables: movie_averageRating <dbl>, budget <dbl>, gross <dbl>,
## #   director_age <dbl>, month <dbl>, day <dbl>, production_season <chr>,
## #   production_day_number <dbl>, director_age_group <chr>, log_gross <dbl>,
## #   log_budget <dbl>, director_freq <chr>
```

Creating dummy variables for genres with a frequency of more than three. Genres with frequency equal to or less than three will be condensed into an "other" group.

```r
# creating table to specify popular genres vs other genres
all_genres <- unlist(strsplit(movie$genres, ","))
genres_table <- data.table(
  genres = all_genres)

genres_table <- genres_table |>
  count(genres, name = "count") |>
```

```r
  mutate(official_genre_list = ifelse(count < 3, "Other", genres),
         other = case_when((official_genre_list == "Other") ~ TRUE,
                           (official_genre_list != "Other") ~ FALSE) )
```

```r
# creating popular genres vs other genres list
popular_genres_df <- filter(genres_table, official_genre_list != "Other")
popular_genre_list <- popular_genres_df$genres

other_df <- filter(genres_table, official_genre_list == "Other")
other_genre_list <- other_df$genres
```

```r
# creating dummy variables for popular genres
for (g in popular_genre_list) {
  movie <- movie |>
    mutate(!!paste0("D_", g) := as.integer(str_detect(genres, g))
           )
}
```

```r
# creating dummy variable for "other"

# initialize D_Other column as 0
movie <- movie |>
  mutate(D_Other = 0)

# D_Other will be set to 1 for all rows that include an "other" genre
for (g in other_genre_list) {
  movie <- movie |>
    mutate(
      D_Other = as.integer(D_Other | str_detect(genres, g))
    )
}

# checking if the dummies were created properly
movie |>
  select(-movie_title, -production_date, -runtime_minutes, -director_name, -birth_year, -movie_averageRa
```

```
## # A tibble: 103 x 16
##    genres        director_freq D_Action D_Adventure D_Biography D_Comedy D_Crime
##    <chr>         <chr>            <int>       <int>       <int>    <int>   <int>
##  1 Action,Adven~ more than th~        1           1           0        0       0
##  2 Adventure,Fa~ more than th~        0           1           0        0       0
##  3 Adventure,Fa~ more than th~        0           1           0        0       0
##  4 Action,Fanta~ more than th~        1           0           0        0       0
##  5 Action,Adven~ more than th~        1           1           0        0       0
##  6 Action,Comed~ three or les~        1           0           0        1       1
##  7 Action,Myste~ more than th~        1           0           0        0       0
##  8 Action,Adven~ more than th~        1           1           0        0       0
##  9 Adventure,Dr~ more than th~        0           1           0        0       0
## 10 Action,Horro~ more than th~        1           0           0        0       0
## # i 93 more rows
## # i 9 more variables: D_Drama <int>, D_Family <int>, D_Fantasy <int>,
## #   D_Horror <int>, D_Mystery <int>, D_Romance <int>, `D_Sci-Fi` <int>,
## #   D_Thriller <int>, D_Other <int>
```

```r
movie
```

```
## # A tibble: 103 x 32
##    movie_title   production_date genres runtime_minutes director_name birth_year
##    <chr>         <date>          <chr>            <dbl> <chr>              <dbl>
##  1 Alita: Battl~ 2019-02-01      Actio~             122 Robert Rodri~       1968
##  2 Alice Throug~ 2016-05-20      Adven~             113 James Bobin         1972
##  3 Harry Potter~ 2005-11-18      Adven~             157 Mike Newell         1942
##  4 Thor          2011-04-29      Actio~             115 Kenneth Bran~       1960
##  5 Gods of Egypt 2016-02-24      Actio~             127 Alex Proyas         1963
##  6 The Green Ho~ 2011-01-14      Actio~             119 Michel Gondry       1963
##  7 I, Robot      2004-07-16      Actio~             115 Alex Proyas         1963
##  8 Starship Tro~ 1997-11-07      Actio~             129 Paul Verhoev~       1938
##  9 Cinderella    2015-03-06      Adven~             105 Kenneth Bran~       1960
## 10 Hollow Man    2000-08-04      Actio~             112 Paul Verhoev~       1938
## # i 93 more rows
## # i 26 more variables: movie_averageRating <dbl>, budget <dbl>, gross <dbl>,
## #   director_age <dbl>, month <dbl>, day <dbl>, production_season <chr>,
## #   production_day_number <dbl>, director_age_group <chr>, log_gross <dbl>,
## #   log_budget <dbl>, director_freq <chr>, D_Action <int>, D_Adventure <int>,
## #   D_Biography <int>, D_Comedy <int>, D_Crime <int>, D_Drama <int>,
## #   D_Family <int>, D_Fantasy <int>, D_Horror <int>, D_Mystery <int>, ...
```

Exploratory Data Analysis

Univariate Distributions

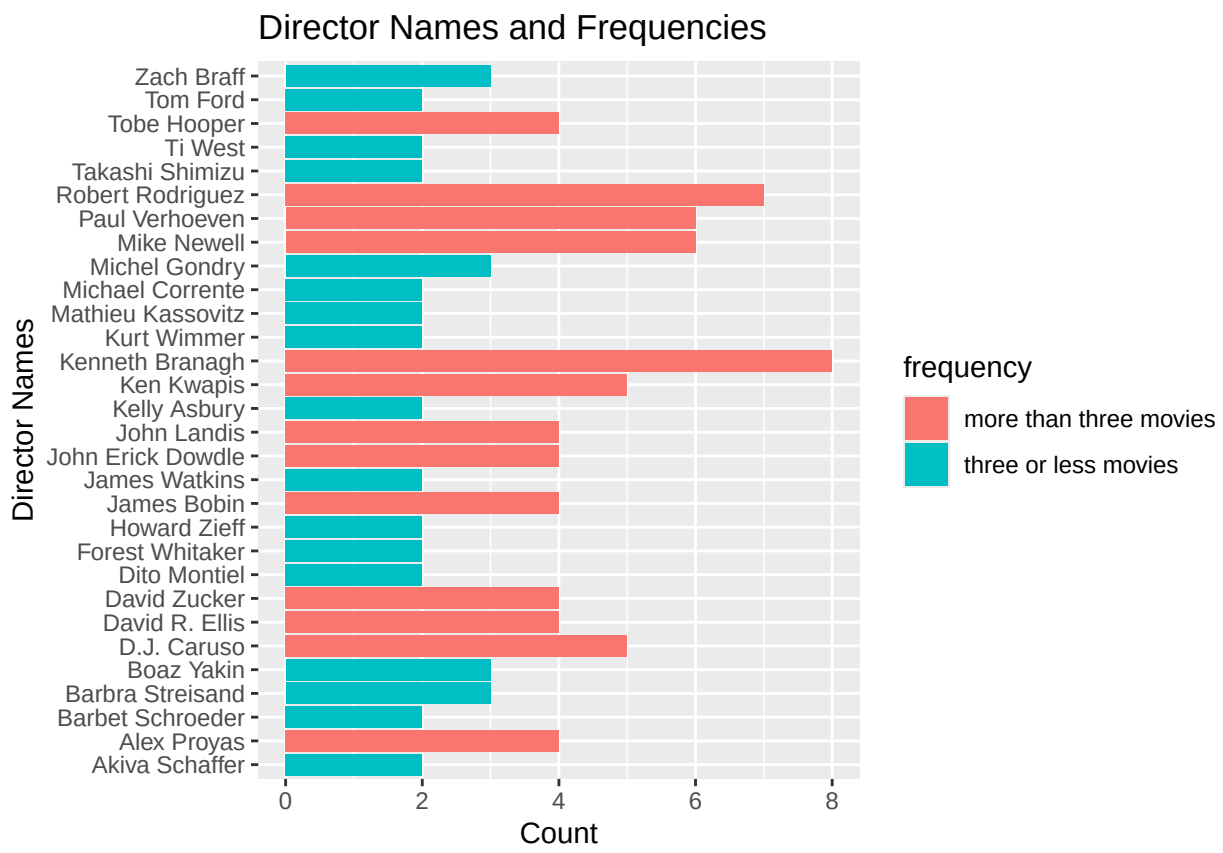
```r
# overall summary: min, mean, median, IQR, max
summary(movie)
```

```
##  movie_title        production_date         genres          runtime_minutes
##  Length:103         Min.   :1977-08-10   Length:103         Min.   : 80.0
##  Class :character   1st Qu.:1998-04-13   Class :character   1st Qu.: 96.0
##  Mode  :character   Median :2006-04-14   Mode  :character   Median :105.0
##                     Mean   :2004-08-21                      Mean   :107.8
##                     3rd Qu.:2012-02-03                      3rd Qu.:115.0
##                     Max.   :2022-03-17                      Max.   :242.0
##  director_name        birth_year   movie_averageRating     budget
##  Length:103         Min.   :1927   Min.   :3.900       Min.   :   600000
##  Class :character   1st Qu.:1947   1st Qu.:5.800       1st Qu.: 13500000
##  Mode  :character   Median :1960   Median :6.400       Median : 27000000
##                     Mean   :1958   Mean   :6.353       Mean   : 37427184
##                     3rd Qu.:1968   3rd Qu.:7.000       3rd Qu.: 45000000
##                     Max.   :1980   Max.   :8.300       Max.   :170000000
##      gross            director_age       month            day
##  Min.   :   102812   Min.   :46.00   Min.   : 1.00   Min.   : 1.00
##  1st Qu.: 25400534   1st Qu.:58.50   1st Qu.: 4.00   1st Qu.: 8.50
##  Median : 58011485   Median :66.00   Median : 7.00   Median :18.00
##  Mean   :101722204   Mean   :67.71   Mean   : 6.67   Mean   :16.58
##  3rd Qu.:137804220   3rd Qu.:79.00   3rd Qu.:10.00   3rd Qu.:25.00
```

```
##  Max.   :886742787   Max.   :99.00   Max.   :12.00   Max.   :31.00
##  production_season  production_day_number director_age_group   log_gross
##  Length:103         Min.   : 2778         Length:103         Min.   :11.54
##  Class :character   1st Qu.:10330         Class :character   1st Qu.:17.05
##  Mode  :character   Median :13252         Mode  :character   Median :17.88
##                     Mean   :12652                            Mean   :17.66
##                     3rd Qu.:15373                            3rd Qu.:18.74
##                     Max.   :19068                            Max.   :20.60
##    log_budget    director_freq         D_Action      D_Adventure
##  Min.   :13.30   Length:103         Min.   :0.000   Min.   :0.0000
##  1st Qu.:16.42   Class :character   1st Qu.:0.000   1st Qu.:0.0000
##  Median :17.11   Mode  :character   Median :0.000   Median :0.0000
##  Mean   :16.96                      Mean   :0.301   Mean   :0.1845
##  3rd Qu.:17.62                      3rd Qu.:1.000   3rd Qu.:0.0000
##  Max.   :18.95                      Max.   :1.000   Max.   :1.0000
##   D_Biography         D_Comedy         D_Crime         D_Drama
##  Min.   :0.00000   Min.   :0.0000   Min.   :0.000   Min.   :0.0000
##  1st Qu.:0.00000   1st Qu.:0.0000   1st Qu.:0.000   1st Qu.:0.0000
##  Median :0.00000   Median :0.0000   Median :0.000   Median :0.0000
##  Mean   :0.04854   Mean   :0.3301   Mean   :0.233   Mean   :0.3883
##  3rd Qu.:0.00000   3rd Qu.:1.0000   3rd Qu.:0.000   3rd Qu.:1.0000
##  Max.   :1.00000   Max.   :1.0000   Max.   :1.000   Max.   :1.0000
##     D_Family         D_Fantasy          D_Horror        D_Mystery
##  Min.   :0.00000   Min.   :0.00000   Min.   :0.0000   Min.   :0.000
##  1st Qu.:0.00000   1st Qu.:0.00000   1st Qu.:0.0000   1st Qu.:0.000
##  Median :0.00000   Median :0.00000   Median :0.0000   Median :0.000
##  Mean   :0.06796   Mean   :0.06796   Mean   :0.1942   Mean   :0.165
##  3rd Qu.:0.00000   3rd Qu.:0.00000   3rd Qu.:0.0000   3rd Qu.:0.000
##  Max.   :1.00000   Max.   :1.00000   Max.   :1.0000   Max.   :1.000
##    D_Romance        D_Sci-Fi        D_Thriller        D_Other
##  Min.   :0.000   Min.   :0.0000   Min.   :0.0000   Min.   :0.00000
##  1st Qu.:0.000   1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:0.00000
##  Median :0.000   Median :0.0000   Median :0.0000   Median :0.00000
##  Mean   :0.165   Mean   :0.1553   Mean   :0.2427   Mean   :0.06796
##  3rd Qu.:0.000   3rd Qu.:0.0000   3rd Qu.:0.0000   3rd Qu.:0.00000
##  Max.   :1.000   Max.   :1.0000   Max.   :1.0000   Max.   :1.00000
```

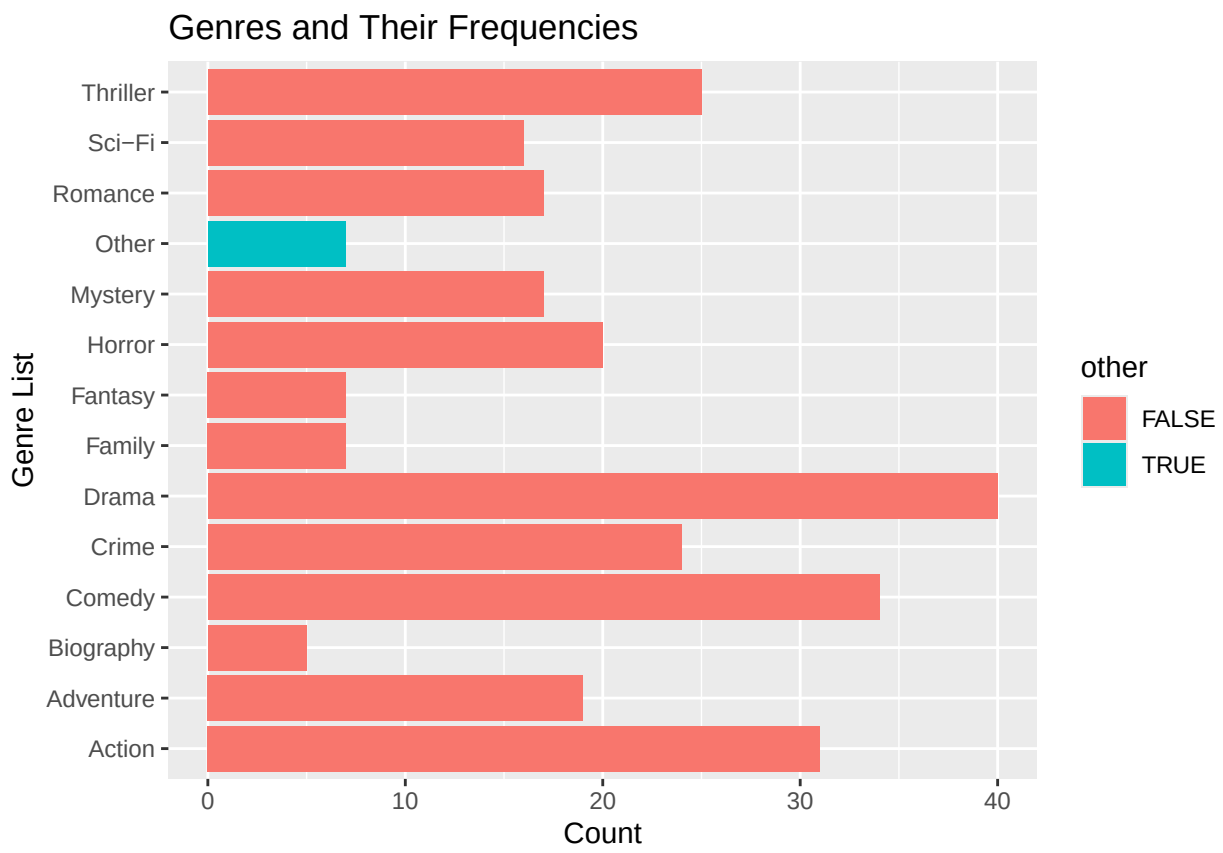
Director Name Distribution

```r
# make a bar graph of the director names and frequencies
ggplot(data = director_freq,
       mapping = aes(x = count, y = director_name)) +
  geom_col(aes(fill = frequency)) +
  labs(x = "Count",
       y = "Director Names",
       title = "Director Names and Frequencies")
```



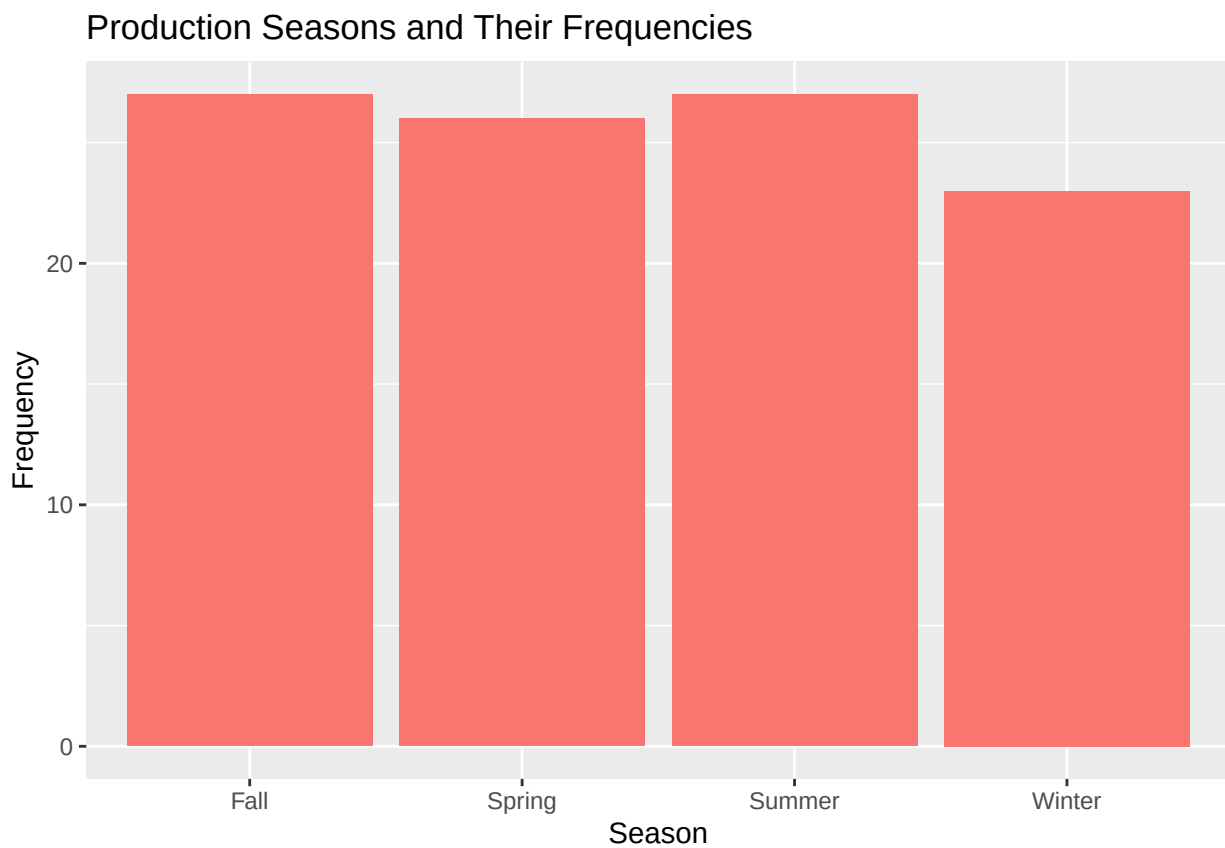
Genres Distribution

```
ggplot(data = genres_table,
       mapping = aes(x = count, y = official_genre_list)) +
  geom_col(aes(fill = other))  +
  labs(x = "Count",
       y = "Genre List",
       title = "Genres and Their Frequencies")+
   theme(
    plot.caption = element_text(hjust = 0)
  )
```



Production Season Distribution

```
ggplot(data = movie,
       mapping = aes(x = production_season, fill = "orange")) +
  geom_bar()  +
  labs(x = "Season",
       y = "Frequency",
       title = "Production Seasons and Their Frequencies"
       ) +
  theme(legend.position = "none")
```



Continuous Variable Distributions and Bivariate Distributions: Scatter-Plot Matrices

```
ggpairs(
  movie |> select(
    runtime_minutes,
    movie_averageRating,
    log_budget,
    log_gross,
    director_age
  ),
  columns = 1:5,
  aes(alpha = 0.6),
  upper = list(
    continuous = wrap(
      "cor",
      digits = 2,
      size = 2.2,
      mapping = aes()
    )
  ),
  lower = list(
    continuous = wrap("points", size = 0.9, color = "coral")
  ),
  diag = list(
```

```
    continuous = wrap("barDiag", bins = 20, fill = "coral")
  ),
  progress = FALSE
)+ labs(caption = "Figure 2: Scatterplot matrix bivariate and univariate distributions") +
   theme(
    plot.caption = element_text(hjust = 0)
  )
```

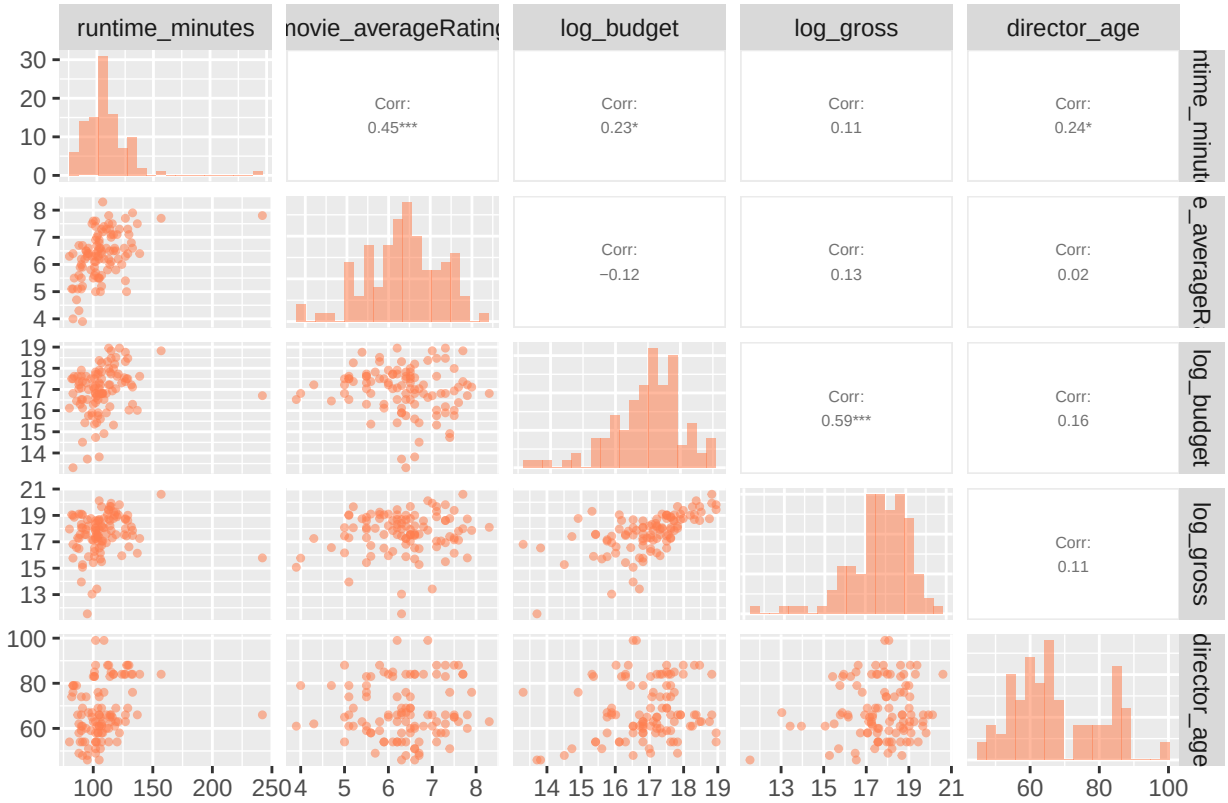


Figure 2: Scatterplot matrix bivariate and univariate distributions

Model Selection

Create a table of relevant variables, then run exhaustive test to find best overall MAIN EFFECTS model with comedy as baseline dummy

```
# exhaustive test to find best MAIN EFFECTS model with comedy as baseline
results_maineffects <- regsubsets(log_gross ~ runtime_minutes + movie_averageRating + production_day_nu
summary(results_maineffects)
```

```
## Subset selection object
## Call: regsubsets.formula(log_gross ~ runtime_minutes + movie_averageRating +
##     production_day_number + production_season + director_age_group +
##     log_budget + director_freq + D_Action + D_Adventure + D_Biography +
##     D_Crime + D_Drama + D_Family + D_Fantasy + D_Horror + D_Mystery +
```

```
##     D_Romance + `D_Sci-Fi` + D_Thriller + D_Other, data = movie,
##     nvmax = 8)
## 26 Variables  (and intercept)
##                                  Forced in Forced out
## runtime_minutes                      FALSE      FALSE
## movie_averageRating                  FALSE      FALSE
## production_day_number                FALSE      FALSE
## production_seasonSpring              FALSE      FALSE
## production_seasonSummer              FALSE      FALSE
## production_seasonWinter              FALSE      FALSE
## director_age_group50s                FALSE      FALSE
## director_age_group60s                FALSE      FALSE
## director_age_group70s                FALSE      FALSE
## director_age_group80s                FALSE      FALSE
## director_age_group90s                FALSE      FALSE
## log_budget                           FALSE      FALSE
## director_freqthree or less movies    FALSE      FALSE
## D_Action                             FALSE      FALSE
## D_Adventure                          FALSE      FALSE
## D_Biography                          FALSE      FALSE
## D_Crime                              FALSE      FALSE
## D_Drama                              FALSE      FALSE
## D_Family                             FALSE      FALSE
## D_Fantasy                            FALSE      FALSE
## D_Horror                             FALSE      FALSE
## D_Mystery                            FALSE      FALSE
## D_Romance                            FALSE      FALSE
## `D_Sci-Fi`                           FALSE      FALSE
## D_Thriller                           FALSE      FALSE
## D_Other                              FALSE      FALSE
## 1 subsets of each size up to 8
## Selection Algorithm: exhaustive
##          runtime_minutes movie_averageRating production_day_number
## 1  ( 1 ) " "             " "                 " "
## 2  ( 1 ) " "             "*"                 " "
## 3  ( 1 ) " "             "*"                 " "
## 4  ( 1 ) " "             "*"                 " "
## 5  ( 1 ) " "             "*"                 " "
## 6  ( 1 ) " "             "*"                 " "
## 7  ( 1 ) " "             "*"                 " "
## 8  ( 1 ) " "             "*"                 " "
##          production_seasonSpring production_seasonSummer
## 1  ( 1 ) " "                     " "
## 2  ( 1 ) " "                     " "
## 3  ( 1 ) " "                     " "
## 4  ( 1 ) " "                     " "
## 5  ( 1 ) " "                     " "
## 6  ( 1 ) " "                     " "
## 7  ( 1 ) " "                     " "
## 8  ( 1 ) " "                     " "
##          production_seasonWinter director_age_group50s director_age_group60s
## 1  ( 1 ) " "                     " "                    " "
## 2  ( 1 ) " "                     " "                    " "
## 3  ( 1 ) " "                     " "                    " "
```

```
## 4  ( 1 ) " "                     " "                   " "
## 5  ( 1 ) " "                     "*"                   " "
## 6  ( 1 ) " "                     "*"                   " "
## 7  ( 1 ) " "                     "*"                   " "
## 8  ( 1 ) "*"                     "*"                   " "
##          director_age_group70s director_age_group80s director_age_group90s
## 1  ( 1 ) " "                   " "                   " "
## 2  ( 1 ) " "                   " "                   " "
## 3  ( 1 ) " "                   " "                   " "
## 4  ( 1 ) " "                   " "                   " "
## 5  ( 1 ) "*"                   " "                   " "
## 6  ( 1 ) "*"                   " "                   " "
## 7  ( 1 ) "*"                   " "                   " "
## 8  ( 1 ) "*"                   " "                   " "
##          log_budget director_freqthree or less movies D_Action D_Adventure
## 1  ( 1 ) "*"        " "                               " "      " "
## 2  ( 1 ) "*"        " "                               " "      " "
## 3  ( 1 ) "*"        " "                               " "      " "
## 4  ( 1 ) "*"        " "                               " "      " "
## 5  ( 1 ) "*"        " "                               " "      " "
## 6  ( 1 ) "*"        " "                               " "      " "
## 7  ( 1 ) "*"        " "                               " "      " "
## 8  ( 1 ) "*"        " "                               " "      " "
##          D_Biography D_Crime D_Drama D_Family D_Fantasy D_Horror D_Mystery
## 1  ( 1 ) " "         " "     " "     " "      " "       " "      " "
## 2  ( 1 ) " "         " "     " "     " "      " "       " "      " "
## 3  ( 1 ) " "         " "     "*"     " "      " "       " "      " "
## 4  ( 1 ) " "         "*"     "*"     " "      " "       " "      " "
## 5  ( 1 ) " "         "*"     " "     " "      " "       " "      " "
## 6  ( 1 ) " "         "*"     " "     " "      " "       " "      " "
## 7  ( 1 ) " "         "*"     "*"     " "      " "       " "      " "
## 8  ( 1 ) " "         "*"     "*"     " "      " "       " "      " "
##          D_Romance `D_Sci-Fi` D_Thriller D_Other
## 1  ( 1 ) " "       " "        " "        " "
## 2  ( 1 ) " "       " "        " "        " "
## 3  ( 1 ) " "       " "        " "        " "
## 4  ( 1 ) " "       " "        " "        " "
## 5  ( 1 ) " "       " "        " "        " "
## 6  ( 1 ) " "       " "        "*"        " "
## 7  ( 1 ) " "       "*"        " "        " "
## 8  ( 1 ) " "       "*"        " "        " "
```

```r
summary(results_maineffects)$bic
```

```
## [1] -35.11345 -36.98408 -42.94314 -42.85727 -42.88992 -41.38292 -38.75771
## [8] -36.69952
```

Using BIC as the information criterion, we find that the third model is the best main effects model, with the predictors being movie_averageRating, log_budget, and D_Drama.

Running exhaustive test including interactions

Narrowing down our predictors to the ones from the main effects model, we will then run an exhaustive search for a sufficient interactions model.

```r
results_interactions <- regsubsets(log_gross ~ movie_averageRating * log_budget * D_Drama, data = movie
summary(results_interactions)
```

```
## Subset selection object
## Call: regsubsets.formula(log_gross ~ movie_averageRating * log_budget *
##     D_Drama, data = movie, nvmax = 8)
## 7 Variables  (and intercept)
##                                        Forced in Forced out
## movie_averageRating                        FALSE      FALSE
## log_budget                                 FALSE      FALSE
## D_Drama                                    FALSE      FALSE
## movie_averageRating:log_budget             FALSE      FALSE
## movie_averageRating:D_Drama                FALSE      FALSE
## log_budget:D_Drama                         FALSE      FALSE
## movie_averageRating:log_budget:D_Drama     FALSE      FALSE
## 1 subsets of each size up to 7
## Selection Algorithm: exhaustive
##          movie_averageRating log_budget D_Drama movie_averageRating:log_budget
## 1  ( 1 ) " "                 "*"        " "     " "
## 2  ( 1 ) "*"                 "*"        " "     " "
## 3  ( 1 ) "*"                 "*"        "*"     " "
## 4  ( 1 ) "*"                 "*"        "*"     "*"
## 5  ( 1 ) "*"                 "*"        "*"     "*"
## 6  ( 1 ) "*"                 "*"        "*"     "*"
## 7  ( 1 ) "*"                 "*"        "*"     "*"
##          movie_averageRating:D_Drama log_budget:D_Drama
## 1  ( 1 ) " "                         " "
## 2  ( 1 ) " "                         " "
## 3  ( 1 ) " "                         " "
## 4  ( 1 ) " "                         " "
## 5  ( 1 ) " "                         "*"
## 6  ( 1 ) "*"                         "*"
## 7  ( 1 ) "*"                         "*"
##          movie_averageRating:log_budget:D_Drama
## 1  ( 1 ) " "
## 2  ( 1 ) " "
## 3  ( 1 ) " "
## 4  ( 1 ) " "
## 5  ( 1 ) " "
## 6  ( 1 ) " "
## 7  ( 1 ) "*"
```

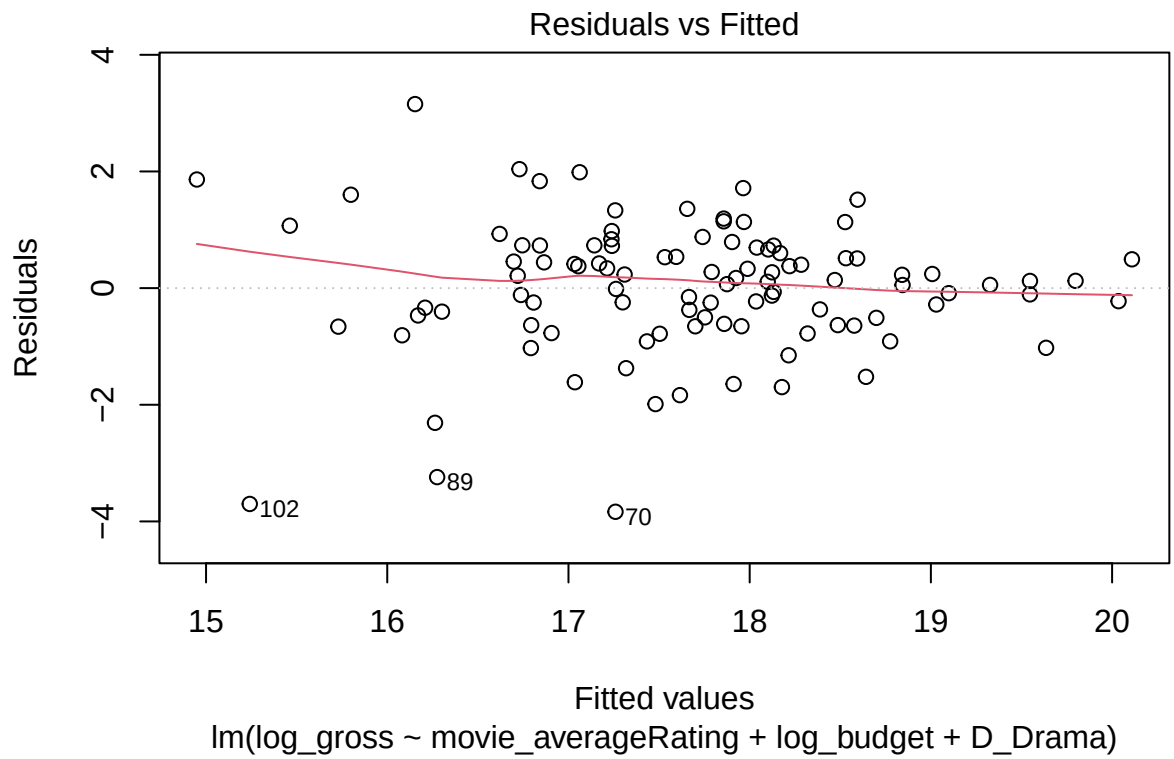
```r
summary(results_interactions)$bic
```

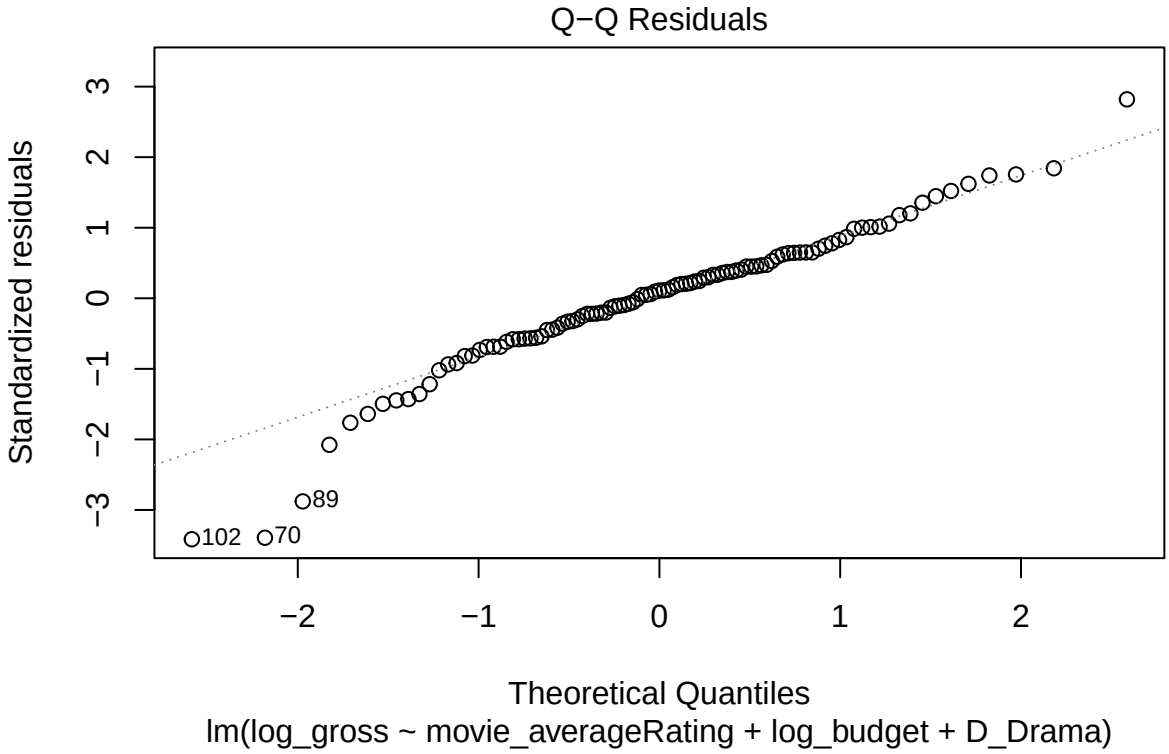
```
## [1] -35.11345 -36.98408 -42.94314 -44.04852 -40.47325 -36.11372 -33.17942
```

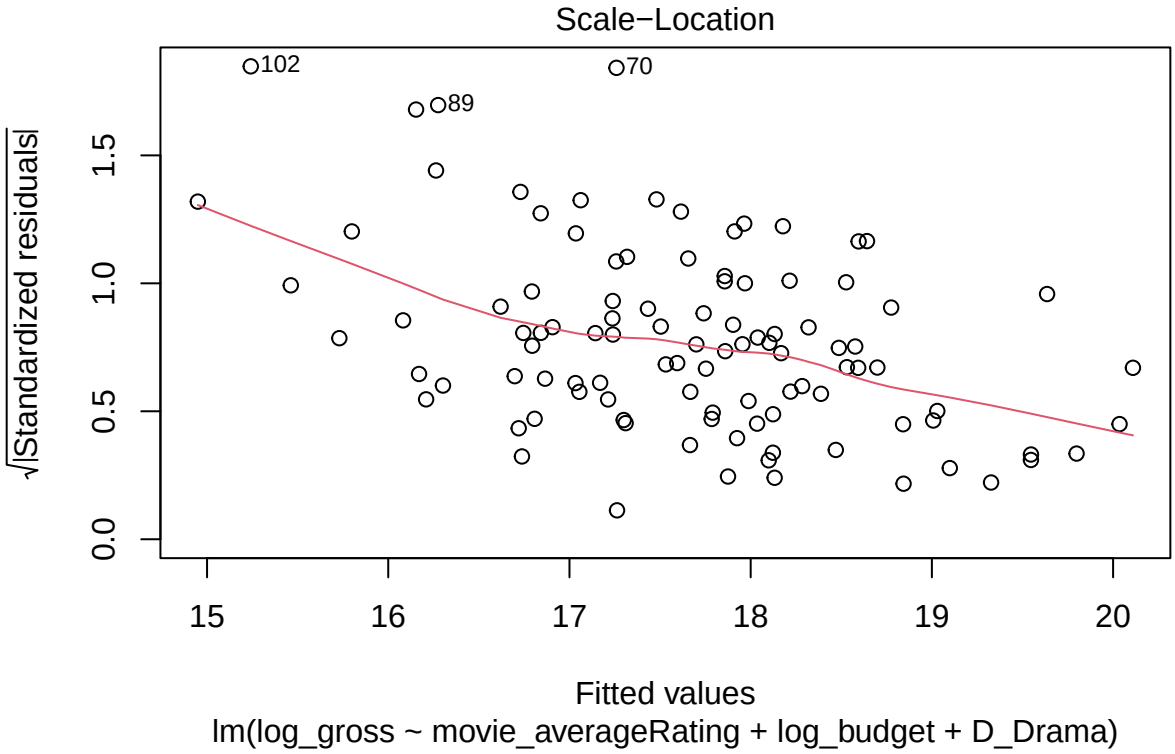
According to the BIC information criterion, the best interactions model is the fourth one, with predictors movie_averageRating + log_budget + D_Drama, and a single interaction, movie_averageRating:log_budget. The second best model is the third model, which is just the main effects model. Our third best is the fifth model, which includes predictors movie_averageRating + log_budget + D_Drama, and two interactions, movie_averageRating:log_budget + log_budget:D_Drama. We will run diagnostics on the top 3 models and compare them using the anova() function.

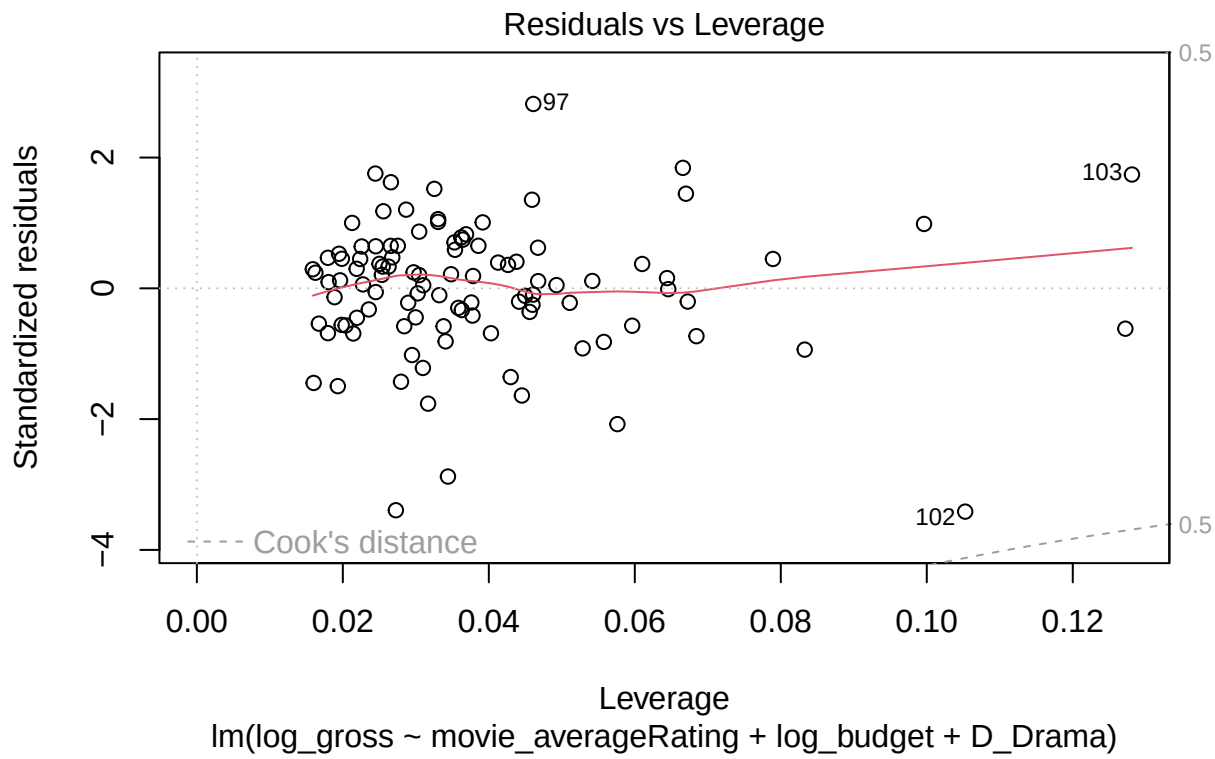
Running diagnostics on top 3 models

```
model1<- lm(log_gross ~ movie_averageRating + log_budget + D_Drama, data = movie)

model2 <- lm(log_gross ~ movie_averageRating + log_budget + D_Drama + movie_averageRating:log_budget, da

model3<- lm(log_gross ~ movie_averageRating + log_budget + D_Drama + movie_averageRating:log_budget + lo

plot(model1)
```

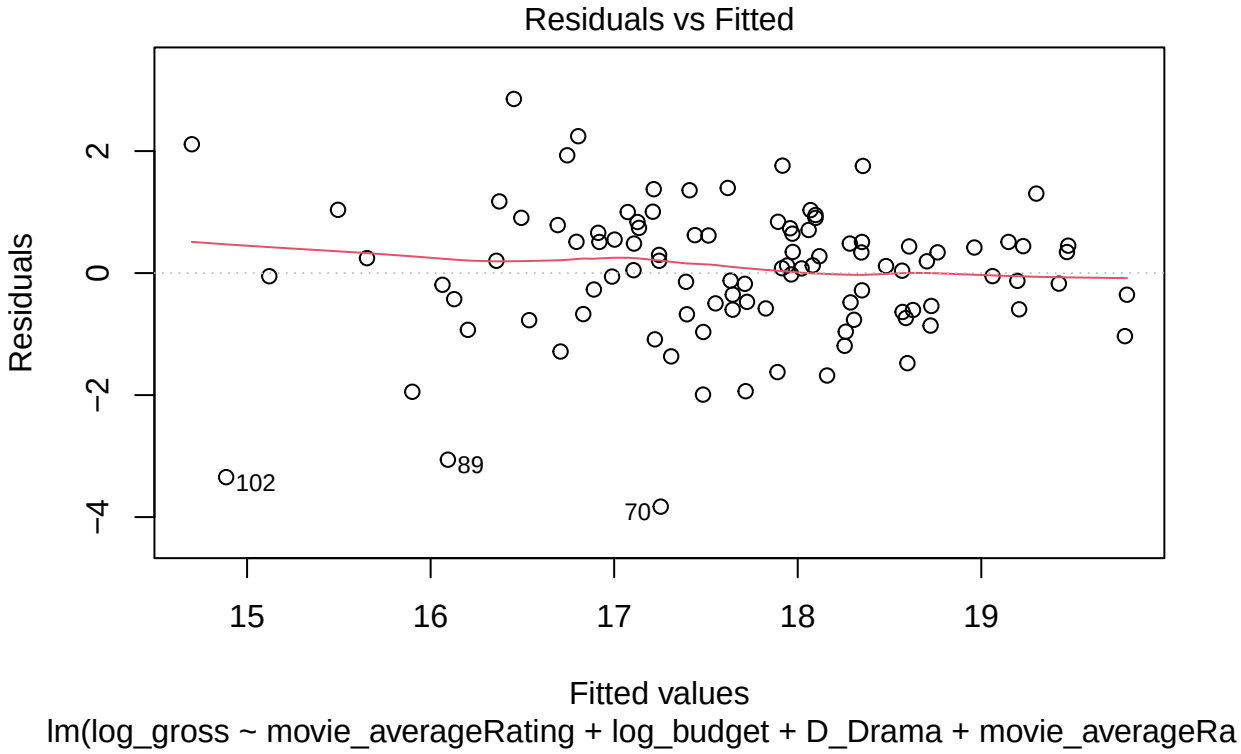


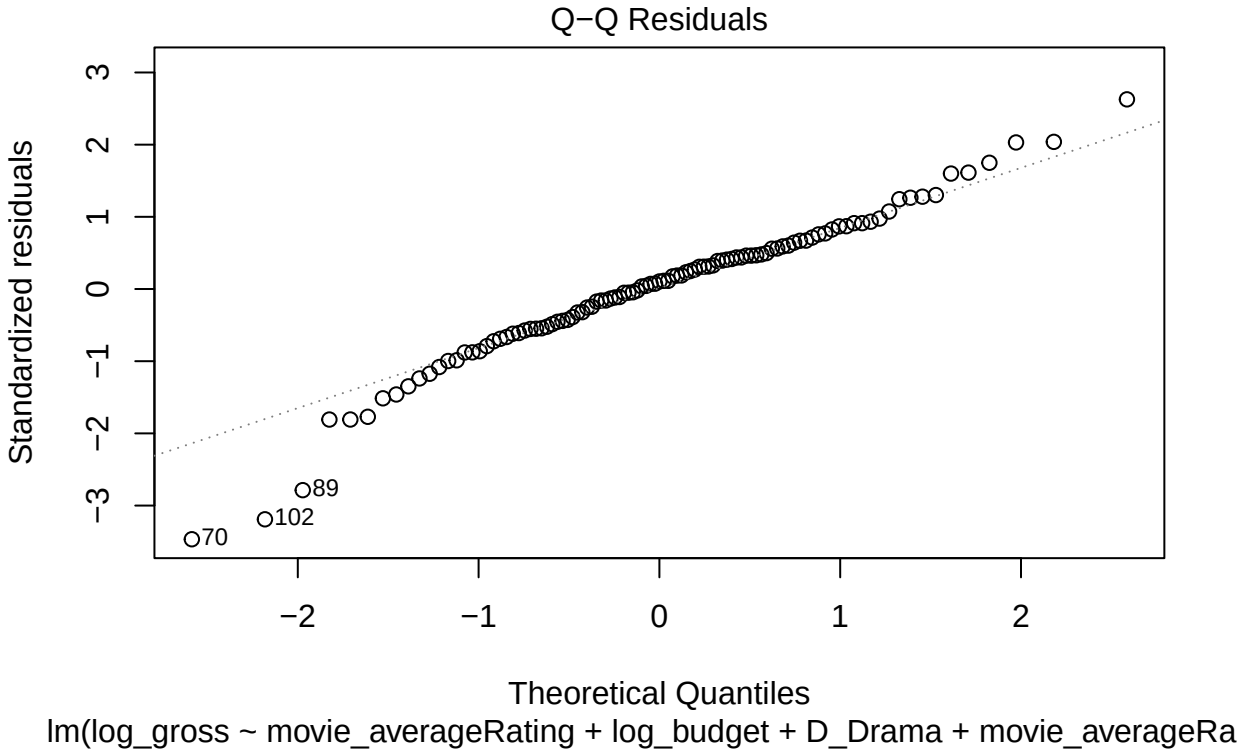

Q−Q Residuals
Standardized residuals
3
2
1
0
−1
−2
−3
89
102
70
−2
−1
0
1
2
Theoretical Quantiles
lm(log_gross ~ movie_averageRating + log_budget + D_Drama)

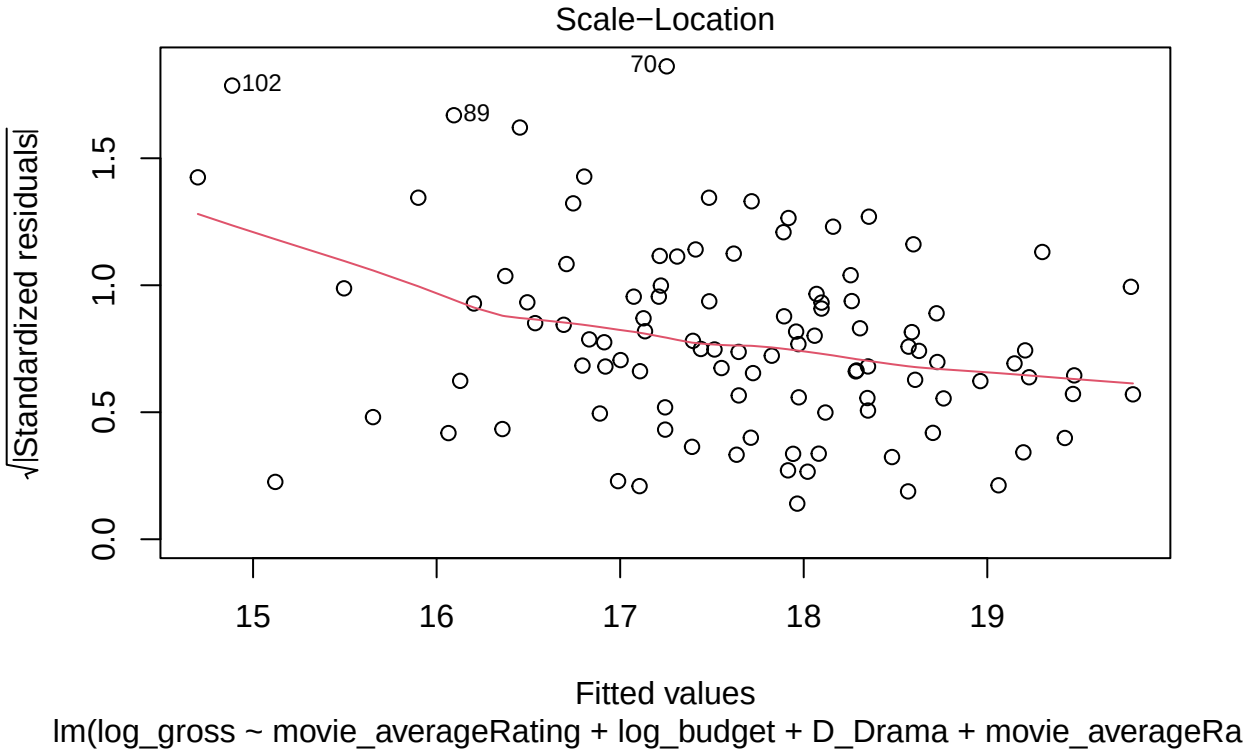

Scale−Location
102
70
89
√|Standardized residuals|
0.0
0.5
1.0
1.5
15
16
17
18
19
20
Fitted values
lm(log_gross ~ movie_averageRating + log_budget + D_Drama)

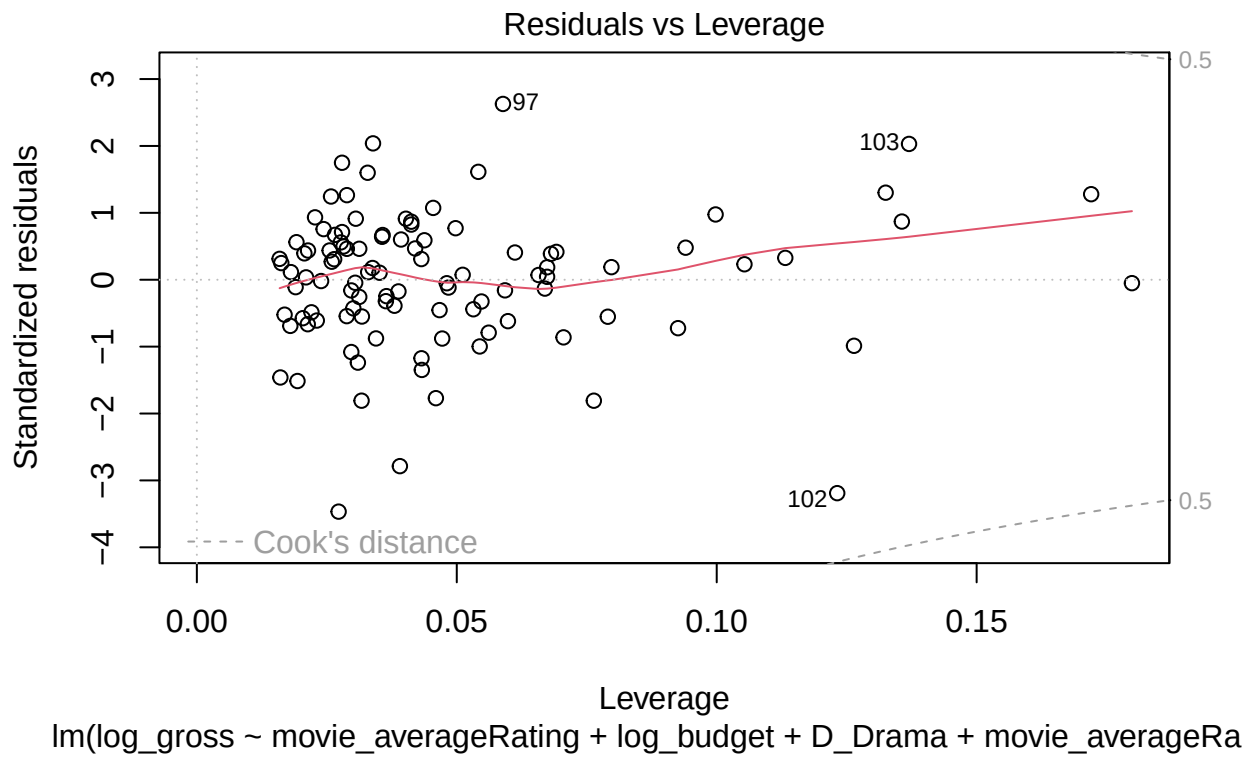



```r
plot(model2)
```

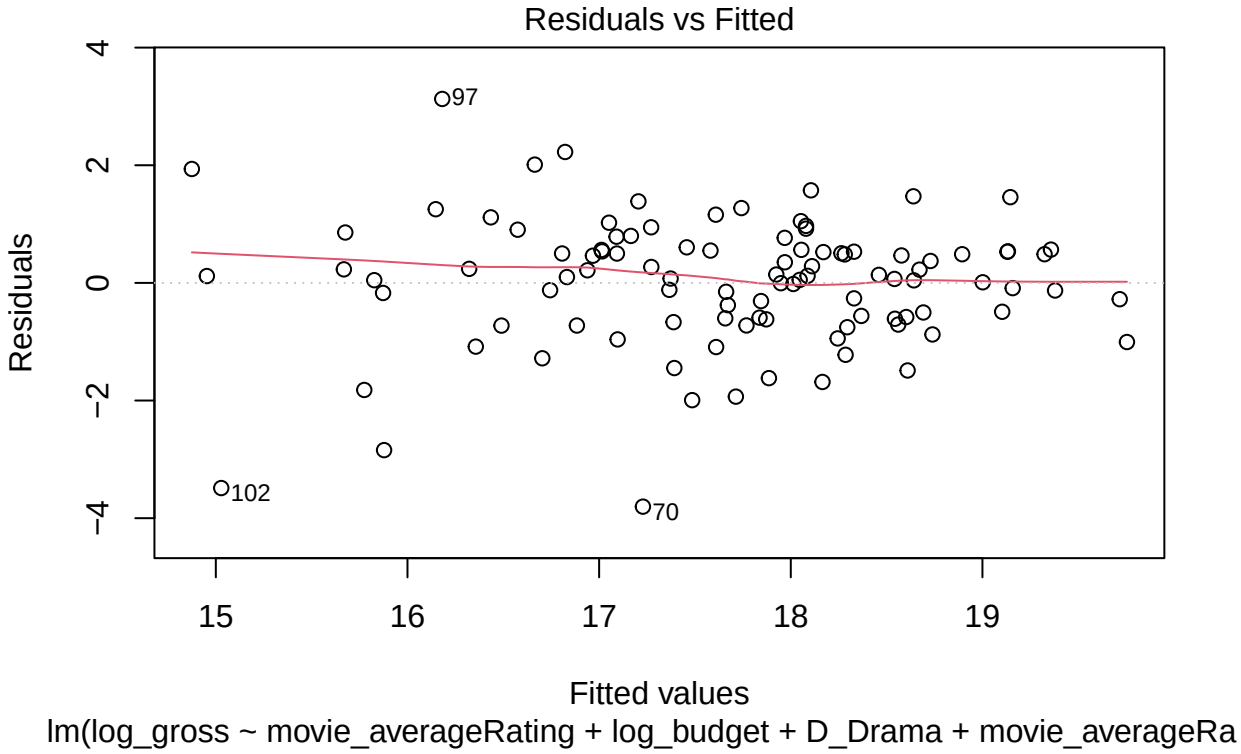

Residuals vs Fitted
Residuals
2
0
−2
−4
102
89
70
15
16
17
18
19
Fitted values
lm(log_gross ~ movie_averageRating + log_budget + D_Drama + movie_averageRa

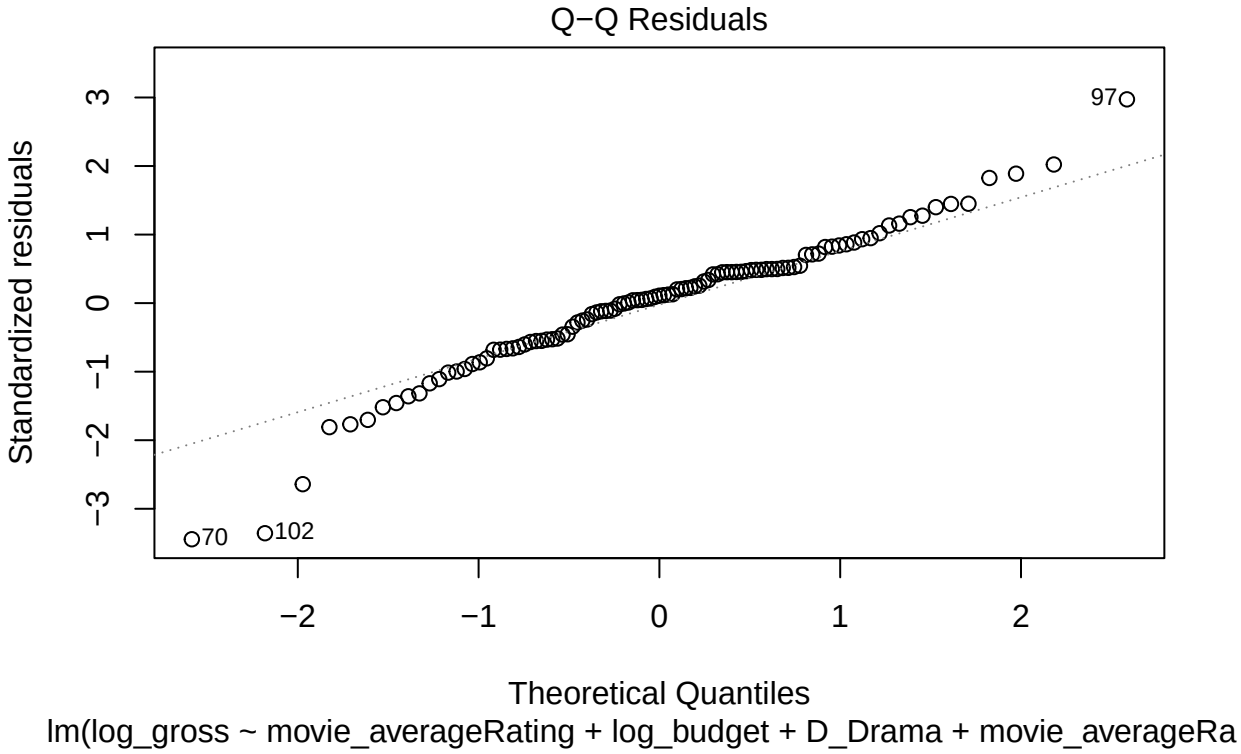

Q−Q Residuals
Standardized residuals
Theoretical Quantiles
lm(log_gross ~ movie_averageRating + log_budget + D_Drama + movie_averageRa
70
102
89
−3
−2
−1
0
1
2
3

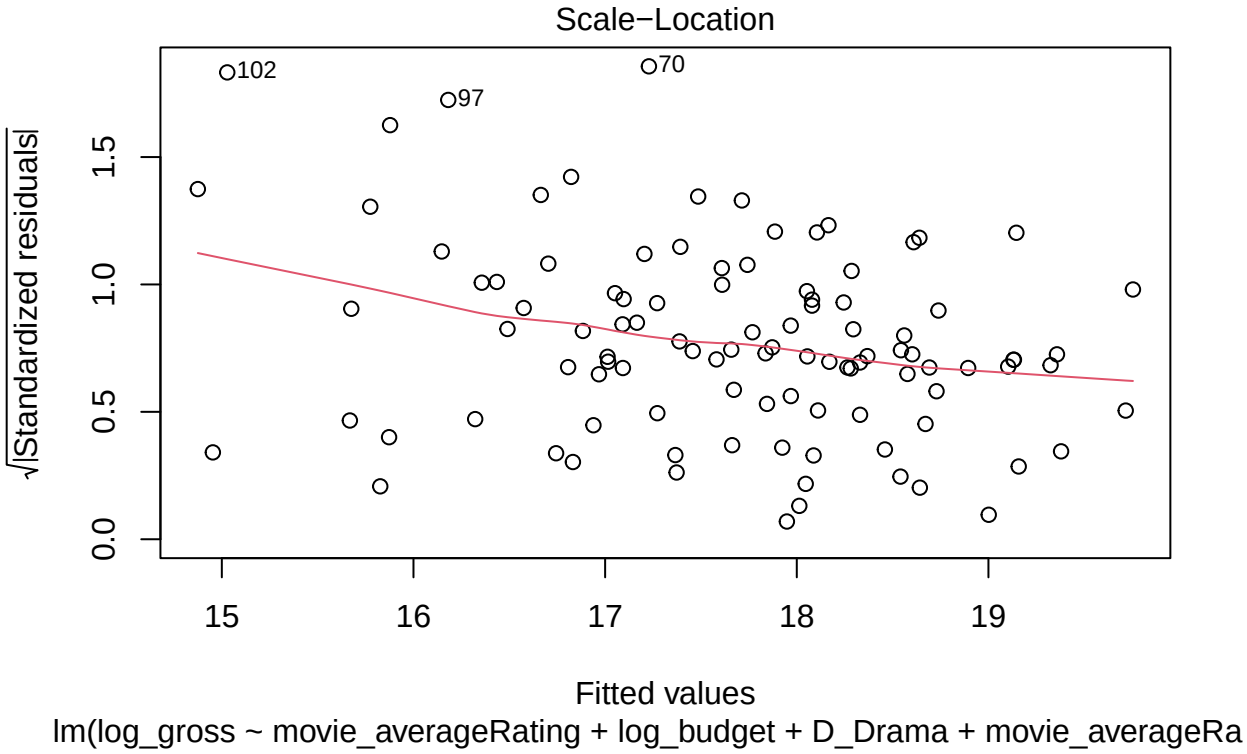

Scale−Location
102
89
70
√|Standardized residuals|
0.0
0.5
1.0
1.5
15
16
17
18
19
Fitted values
lm(log_gross ~ movie_averageRating + log_budget + D_Drama + movie_averageRa

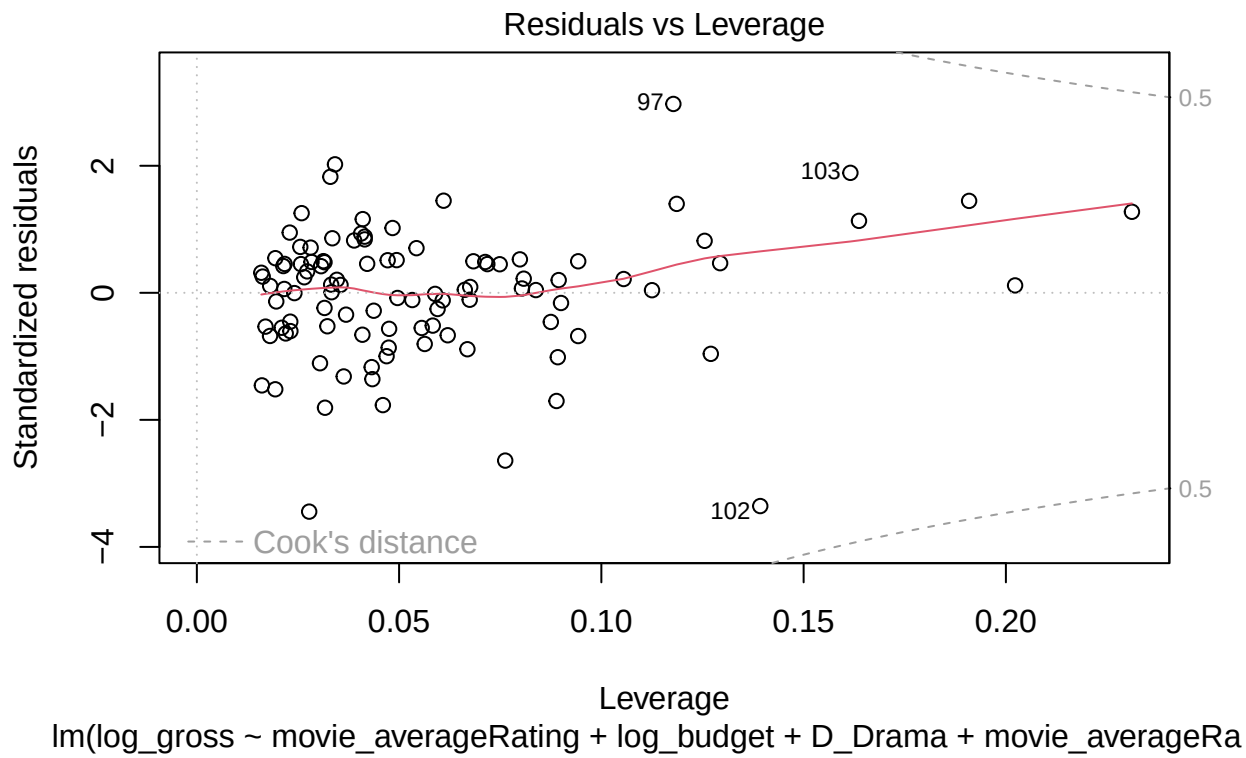



```r
plot(model3)
```


Residuals vs Fitted
Residuals
Fitted values
lm(log_gross ~ movie_averageRating + log_budget + D_Drama + movie_averageRa
97
102
70
4
2
0
−2
−4
15
16
17
18
19


Q−Q Residuals
Standardized residuals
Theoretical Quantiles
lm(log_gross ~ movie_averageRating + log_budget + D_Drama + movie_averageRa
97
70
102
−3
−2
−1
0
1
2
3


Scale−Location
102
97
70
√|Standardized residuals|
0.0
0.5
1.0
1.5
15
16
17
18
19
Fitted values
lm(log_gross ~ movie_averageRating + log_budget + D_Drama + movie_averageRa



Test the remaining two models with F-test

According to the diagnostics, we see that each of the linear assumption has been met. In other words, the predictors exhibit a linear relationship with the response variable.

Additionally, all 3 qqplots have relatively similar layouts, with most of the points except 3 anomalies closely following the 45 degree line. This suggests an approximately normal distribution of the error terms.

The test for nonconstant variance is most violated by model1, the model with only main effects. The standardized residual values appear to be negatively correlated with the fitted values, exhibiting decreasing variance for larger fitted values. However, the slope is flatter for the other two models. The points are more spread out and exhibit less of a pattern, better supporting the assumption of nonconstant variance.

Due to the results of the diagnostics, we will now only compare model2 and model3.

```
#anova(reduced, full)

anova(model2, model3)
```

```
## Analysis of Variance Table
##
## Model 1: log_gross ~ movie_averageRating + log_budget + D_Drama + movie_averageRating:log_budget
## Model 2: log_gross ~ movie_averageRating + log_budget + D_Drama + movie_averageRating:log_budget +
##     log_budget:D_Drama
##   Res.Df    RSS Df Sum of Sq      F Pr(>F)
## 1     98 122.90
## 2     97 121.64  1    1.2576 1.0029 0.3191
```

Since the p value is high, we fail to reject the null hypothesis that the first model is a more fitting model in predicting log_gross. Thus, we will choose model2 instead of model3.

Final Model Report

```r
final_model <- model2
summary(final_model)
```

```
##
## Call:
## lm(formula = log_gross ~ movie_averageRating + log_budget + D_Drama +
##     movie_averageRating:log_budget, data = movie)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.8287 -0.5972  0.1151  0.6342  2.8546
##
## Coefficients:
##                                Estimate Std. Error t value Pr(>|t|)
## (Intercept)                    -42.5426    18.5058  -2.299 0.023636 *
## movie_averageRating              7.0902     2.8072   2.526 0.013150 *
## log_budget                       3.3865     1.0835   3.126 0.002335 **
## D_Drama                         -0.8252     0.2334  -3.536 0.000623 ***
## movie_averageRating:log_budget  -0.3898     0.1645  -2.370 0.019751 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.12 on 98 degrees of freedom
## Multiple R-squared:  0.4793, Adjusted R-squared:  0.4581
## F-statistic: 22.55 on 4 and 98 DF,  p-value: 3.165e-13
```

```r
par(
  mfrow = c(3,1),
  mar = c(4, 4, 3, 1),
  oma = c(4, 0, 0, 0)
)

plot(final_model, which = 1:3)

mtext(
  "Figure 3: Linear model diagnostic plots",
  side = 1,
  outer = TRUE,
  line = 1,
  adj = 0
)
```

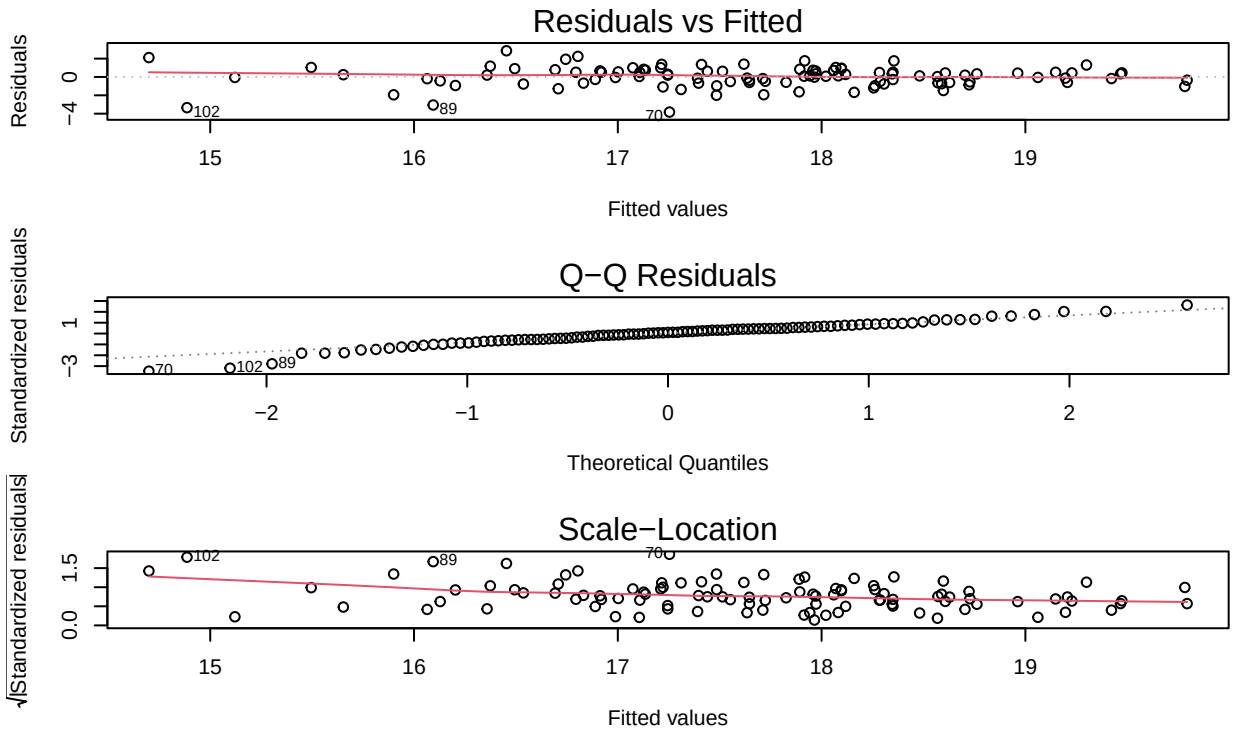


Figure 3: Linear model diagnostic plots

Our final model is:

$$log_gross_i = -42.5426 + 7.0902 MaR_i + 3.3865 log_budget_i - 0.8252 D_{DRAMAi} - 0.3898 (MaR_i * log_budget_i) + e_i, \quad e_i \sim N(0, \sigma^2$$

Model Interpretation:

```
library(tibble)
library(gt)
```

```
## Warning: package 'gt' was built under R version 4.4.3
```

```
exp(-0.825)
```

```
## [1] 0.438235
```

```
100 * (exp(log(1.01)*3.39) - 1)
```

```
## [1] 3.430698
```

```
3.3865 * 0.01
```

```
## [1] 0.033865
```

```r
1 - exp(-0.8252)
```

```
## [1] 0.5618526
```

```r
coef_table <- tibble(
  Coefficient = c("B0 (Intercept)",
                  "B1: Movie Average Rating",
                  "B2: ln(budget)",
                  "B3: Drama",
                  "B4: Movie Average Rating × ln(budget)"),

  Estimate = c(-42.54, 7.09, 3.39, -0.83, -0.39),

  `ln(gross) interpretation` = c(
    "Expected ln(gross) for a non-Drama film with rating = 0 and budget = $1.",
    "For non-Drama films, a 1-point increase in rating increases ln(gross) by 7.09, before accounting fo
    "For non-Drama films at rating = 0, a 1% increase in budget raises ln(gross) by 0.0339.",
    "Drama films have ln(gross) 0.83 lower than non-Drama films, holding rating and budget constant.",
    "As budget increases, the positive effect of rating on ln(gross) decreases."
  ),

  `Gross revenue interpretation` = c(
    "No meaningful real-world interpretation; serves as a baseline from which other effects adjust rever
    "At a given budget, higher ratings substantially increase revenue, but the size of the effect depend
    "A 1% increase in budget leads to about a 3.4% increase in gross revenue, holding movie rating const
    "Drama films earn about 56% less gross revenue than non-Drama films on average.",
    "Big-budget films gain less additional revenue from higher ratings than smaller-budget films."
  )
)

coef_table |>
  gt() |>
  tab_header(
    title = "Interpretation of Regression Coefficients",
    subtitle = "Response Variable: ln(gross)"
  ) |>
  fmt_number(
    columns = Estimate,
    decimals = 2
  ) |>
  cols_align(
    align = "center",
    columns = Estimate
  ) |>
  tab_style(
    style = cell_text(weight = "bold"),
    locations = cells_column_labels()
  ) |>
  tab_source_note(
    source_note = "Figure 4: Regression Coefficients and Interpretations"
  )
```

Interpretation of Regression Coefficients

Response Variable: ln(gross)

Coefficient	Estimate	ln(gross) interpretation
B0 (Intercept)	-42.54	Expected ln(gross) for a non-Drama film with rati
B1: Movie Average Rating	7.09	For non-Drama films, a 1-point increase in rating
B2: ln(budget)	3.39	For non-Drama films at rating = 0, a 1% increase
B3: Drama	-0.83	Drama films have ln(gross) 0.83 lower than non-D
B4: Movie Average Rating × ln(budget)	-0.39	As budget increases, the positive effect of rating o

Figure 4: Regression Coefficients and Interpretations

Key Question: Is an increased budget justified?

```r
# is the beta AT LEAST one?
# null: beta = 1
# alt: beta > 1
est <- 3.3865
null <- 1
se <- 1.0835
alpha <- 0.05
test_stat <- (est - null) / se
pt(test_stat, 98, lower.tail = FALSE)
```

```
## [1] 0.01498408
```

```r
confint(final_model, level = 0.95)
```

```
##                                      2.5 %      97.5 %
## (Intercept)                    -79.2667805 -5.81835804
## movie_averageRating              1.5193718 12.66110913
## log_budget                       1.2363446  5.53661612
## D_Drama                         -1.2883934 -0.36206318
## movie_averageRating:log_budget  -0.7162129 -0.06339948
```

At significance level 0.05, we have enough evidence to reject our null and claim that the coefficient for log_budget is greater than 1. The same results can also be observed from constructing a 95% confidence interval of the coefficient for log_budget, which ranges between 1.2363446 and 5.53661612, all values greater than 1.

Prediction

Predict the revenue generated by a new film with the following characteristics: production date: June 15, 2013, run time: 2 hrs, rating: 7.0, genre: comedy, budget: 10 million

```r
log(10000000)
```

```
## [1] 16.1181
```

```r
pred_int<- predict(final_model, newdata = data.frame(
    movie_averageRating = 7.0,
    log_budget = 16.1181,
    D_Drama = 0),
  interval = "prediction",
  level = 0.95
)

exp(pred_int) #applying exp() to our prediction interval, since we did a log transform
```

```
##        fit     lwr       upr
## 1 48263942 5044977 461728150
```